Lattice Types: A Research Note

Towards a Lattice-Theoretic Foundation for Behavioral Type Systems

Alexandre Zua Caldeira
LASIGE/FCUL, University of Lisbon

`draft -- not for distribution`

March 2026

Abstract

We observe that the lattice structure recently established for session type state spaces may not be an isolated property of session types, but rather an instance of a more general phenomenon: several families of behavioral type systems have semantic domains that form lattices. We introduce the notion of *lattice type* as a type equipped with a finite lattice of behavioral states, and sketch how session types, typestate, information-flow types, and CRDT types can be uniformly viewed as lattice types. We pose the key open questions—completeness, synthesis, and the role of the parallel constructor—and outline a potential research programme. This note is preliminary and exploratory; no new theorems are proved.

Status. Internal research note. Records an observation arising from work on session types as algebraic reticulates [1]. The purpose is to document the idea precisely enough to evaluate its feasibility and scope before committing to a full development.

1 Motivation

Our recent work establishes that the state space $L(S)$ of a session type S, ordered by reachability, forms a lattice [1]. The parallel constructor $S_1 \| S_2$ plays a distinguished role: its state space is the product $L(S_1) \times L(S_2)$, and the product of lattices is a lattice. This is not merely a nice property—it is what *forces* lattice structure in the presence of concurrency.

A natural question arises:

> *Is lattice structure specific to session types, or is it a common feature of behavioral type systems?*

If the latter, then the lattice is not a *consequence* of session type structure but a *reason* why behavioral types work. Session types would be one syntax for a more general class of types whose semantic invariant is lattice structure. This note explores this possibility.

2 Lattice Types: Definition

We propose the following abstract notion.

Definition 1 (Lattice type)**.** *A* lattice type *is a triple* $(T, L_T, \leq_T)$ *where:*

(i) T *is a type expression in some type language,*

(ii) L_T *is a finite set of* behavioral states*,*

(iii) $(L_T, \leq_T)$ *is a bounded lattice with top* $\top_T$ *and bottom* $\bot_T$*,*

such that:

- $\top_T$ *represents the* most permissive *behavioral state (initial, all capabilities available),*
- $\bot_T$ *represents the* most restricted *behavioral state (terminal, no capabilities remain),*
- *the ordering* $s_1 \leq_T s_2$ *means that* s_1 *is a behavioral refinement of* s_2 *(fewer capabilities, more committed).*

The lattice operations have uniform behavioral meaning:

Definition 2 (Behavioral meet and join)**.** *For behavioral states* $s_1, s_2 \in L_T$*:*

- $s_1 \sqcap s_2$ *is the* intersection of capabilities*—the most permissive state reachable from both* s_1 *and* s_2*.*
- $s_1 \sqcup s_2$ *is the* union of capabilities*—the least permissive state from which both* s_1 *and* s_2 *are reachable.*

Remark 1. *Definition 1 is deliberately abstract. It does not prescribe a syntax, a transition system, or a specific type discipline. The claim is that this structure is the common semantic invariant across multiple type systems.*

3 Candidate Instantiations

We survey several behavioral type disciplines and argue informally that each can be viewed as a lattice type. We do not prove these claims here; each would require its own formal development.

3.1 Session types (Sess)

This is our established result. Given a session type S, the state space $L(S)$ with reachability ordering forms a bounded lattice. The constructors map to lattice operations as follows:

Constructor	**Lattice role**	**Operation**
$\&\{m_1 : S_1, \ldots, m_n : S_n\}$	branching	generates upper structure
$\oplus\{l_1 : S_1, \ldots, l_n : S_n\}$	selection	generates upper structure
$S_1 \| S_2$	parallel	product lattice $L(S_1) \times L(S_2)$
$\mu X.S$	recursion	SCC quotient preserves lattice
end	terminal	bottom element $\bot$

The parallel constructor is the key: it is the only constructor that *necessarily* produces lattice structure (products of lattices are lattices). Without it, state spaces are typically chains or trees, which are lattices only trivially.

3.2 Typestate (TS)

Typestate systems [2, 3] track object lifecycle as a finite set of states with permitted method calls. The permission ordering—"state s_1 has at least the permissions of s_2"—is a partial order.

Conjecture 1. *For typestate systems with a distinguished initial state and a distinguished final state, the permission poset forms a lattice when the typestate graph satisfies conditions analogous to our session type well-formedness.*

The connection to session types is direct: Mungo [4] and BICA [5] implement session types as typestate on objects. Our lattice result for session types therefore already covers a significant fragment of typestate.

3.3 Information-flow types (IF)

Security type systems for information flow [6, 7] assign security levels to data, ordered by "may flow to." The security levels form a lattice by construction—this is Denning's original insight. The lattice operations are:

- $\sqcap$ = greatest lower bound of security levels (declassification boundary),
- $\sqcup$ = least upper bound (combination of classified data).

Information-flow types are lattice types *by design*, but the connection to behavioral state has not been made explicit. A security-typed channel has a behavioral state space (what flows are permitted at each program point) that inherits the lattice structure of the security levels.

3.4 CRDT types (CRDT)

Conflict-free replicated data types [8] require that concurrent updates converge via a join-semilattice. The semilattice ensures that the merge of any two replica states is well-defined and commutative.

Remark 2. *CRDTs require only a join-semilattice (not a full lattice), so they represent a* weaker *instantiation of the lattice type concept. Extending them to full lattice types would require a meaningful meet operation, which may not always exist in the CRDT setting.*

3.5 Summary

Discipline	States	Top	Bottom	Lattice?
Session types	protocol states	initial	end	yes (proved)
Typestate	permissions	all-permitted	finalized	conjectured
Info-flow	security levels	public	top-secret	yes (by design)
CRDTs	replica states	initial	—	semilattice

4 The Synthesis Direction

If lattice types are the semantic foundation, a natural question is *synthesis*: given a lattice, can we produce a session type (or other behavioral type) whose state space is that lattice?

Open Question 1 (Completeness)**.** *Is every finite bounded lattice $(L, \leq)$ with labeled edges isomorphic to $L(S)$ for some session type S?*

Open Question 2 (Canonical form)**.** *If multiple session types $S_1, S_2, \ldots$ have isomorphic state spaces, is there a canonical representative? What is the equivalence relation?*

Open Question 3 (Decomposition)**.** *Can a lattice be algorithmically decomposed into:*

- *product components (corresponding to $\|$),*
- *chain components (corresponding to sequential branches),*
- *cyclic components (corresponding to μ),*

yielding a session type expression by construction?

A positive answer to Question 1 would establish that session types are *complete* for finite lattice types: every lattice type has a session type syntax. This would be a strong foundational result.

Remark 3. *We suspect the answer to Question 1 is* no *in general. Not every finite lattice decomposes into products and chains. The free lattice on 3 generators, for instance, has 18 elements and a structure that does not obviously correspond to any session type. Characterizing the* realizable *lattices is likely the right question.*

5 Relationship to Current Work

This note suggests a potential generalization of our session type lattice result, but it is important to be precise about the relationship.

What we have proved.
Session type state spaces form lattices. The parallel constructor produces product lattices. The recursion quotient preserves lattice structure. These are concrete, verified results with tool support [1].

What this note proposes.
That the lattice property is not specific to session types but is a common semantic invariant of behavioral type systems. This is an *observation*, not a theorem.

What would need to be proved.
Each candidate instantiation (typestate, information-flow, CRDTs) would require its own formal development: defining the state space, establishing the ordering, and proving lattice properties. The typestate case is likely the most tractable, given its proximity to session types.

Strength comparison.
If the programme succeeds, it would be strictly stronger than our current result. "Session type state spaces are lattices" would become a corollary of "behavioral type state spaces are lattice types, and session types are one instance." The parallel constructor's role would generalize to: *any type constructor that introduces independent concurrent state produces product lattice structure.*

6 Risks and Honest Assessment

We note the following risks:

1. **Vacuity.** If the definition of lattice type is too abstract, it may be trivially satisfied and say nothing interesting. The definition must be constrained enough that lattice structure is a *non-obvious* property, not a definitional artifact.

2. **Incompleteness.** The synthesis direction (Question 1) may fail, meaning that session types capture only a *proper subclass* of lattice types. This is not fatal—the characterization of the realizable fragment is itself interesting—but it weakens the foundational claim.

3. **Semilattice gap.** CRDTs and some other concurrent systems naturally produce semilattices, not full lattices. If the framework requires full lattices, it may exclude important applications. A semilattice generalization may be needed.

4. **Scope.** The information-flow case is lattice-by-design (Denning's model), so including it may be seen as circular rather than illuminating. The genuine test cases are typestate and any system where lattice structure is *emergent* rather than stipulated.

7 Proposed Next Steps

We suggest a cautious, incremental approach:

1. **Typestate lattice theorem.** Prove that typestate permission posets form lattices under appropriate well-formedness conditions. This is the closest generalization of our session type result and likely requires the least new machinery.

2. **Realizability.** Characterize which finite lattices are realizable as session type state spaces. This addresses Question 1 and clarifies the expressive power of session types within the lattice type framework.

3. **Product decomposition.** Investigate when a lattice admits a non-trivial product decomposition and whether this decomposition corresponds to a parallel constructor in the type syntax (Question 3).

4. **Morphism transfer.** Check whether the morphism hierarchy (isomorphism, embedding, projection, Galois connection) from our session type work transfers to the general lattice type setting. If so, it would provide a uniform theory of behavioral refinement across type disciplines.

5. **Deferred: semilattice generalization.** Only pursue this if the full-lattice framework proves too restrictive for applications of interest.

8 Conclusion

The observation that session type state spaces form lattices invites a broader question: whether lattice structure is a common invariant of behavioral type systems. If so, the notion of *lattice type* could serve as a unifying semantic foundation, with session types as one syntactic instantiation.

This note does not answer that question. It records the observation, defines the concept precisely enough to evaluate, identifies the key open questions, and proposes a research path. The next concrete step is the typestate lattice theorem, which would provide the first evidence beyond session types.

The potential payoff is significant: "session type state spaces are lattices" is a property result; "behavioral type systems are lattice types" is a foundational one. Whether the generalization holds remains to be seen.

References

[1] A. Zua Caldeira. Session type state spaces form lattices. Draft, 2026.

[2] R. E. Strom and S. Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Trans. Softw. Eng.*, 12(1):157–171, 1986.

[3] R. DeLine and M. Fähndrich. Typestates for objects. In *ECOOP*, pages 465–490, 2004.

[4] D. Kouzapas, O. Dardha, R. Perera, and S. J. Gay. Typechecking protocols with Mungo and StMungo. In *PPDP*, pages 146–159, 2018.

[5] J. Campos and V. T. Vasconcelos. Channels as objects in concurrent object-oriented programming. In *PLACES*, 2009.

[6] D. E. Denning. A lattice model of secure information flow. *Commun. ACM*, 19(5):236–243, 1976.

[7] A. Sabelfeld and A. C. Myers. Language-based information-flow security. *IEEE J. Sel. Areas Commun.*, 21(1):5–19, 2003.

[8] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. In *SSS*, pages 386–400, 2011.